Reddit Clone
React viva guide

Updated for the supplied five-criterion React UI rubric. Learn the theory, explain your actual code, demonstrate the features and reason about failures.

Project and source snapshot

github.com/Yazir-Musthafa/Reddit-Clone

Reviewed code: 60e7ea9. **Prepared:** 27 October 2026. File names, numbered extracts and source links refer to this exact commit. This edition replaces the earlier Module V guide.

What changed for this rubric

• Reusable JSX components now render full and compact post cards, menus, vote controls, membership controls and media.

• A real immutable post reducer replaces prop mutation and the old forced-refresh counter.

• Settings is now composed of controlled React account, profile and preference forms with reusable fields, validation and submission handling.

• Settings tabs use React Router search parameters; Back, Forward and reload preserve the selected screen.

• Mobile grid, editor toolbar and tablet press-asset overflow are fixed. Dynamic profile colors remain visually consistent.

Evidence and limits

20 browser tests and 4 unit tests passed. All 11 page entries were checked at 390, 768 and 1440 pixels. **52 baseline screenshot comparisons reported zero changed pixels.** The responsive fixes improve additional narrow states not present in that baseline.

The guide maps implementation evidence to the rubric; it does not award marks. The evaluator decides the score. The fifth criterion requires your own explanation, debugging and truthful account of your contribution.

This remains a hybrid React frontend: core post/settings/forms are declarative React; some other screens retain template-backed controllers. Long source lines wrap visually; blank line-number gutters indicate continuation.

The exact rubric: evidence for each criterion

Criterion	Excellent-level target in the image	Project evidence
1. Component design & rendering	Reusable, structured components; effective props; correct conditional rendering.	PostContent components; shared FormSection/FormField; callback props; stable keys; login/menu/media/status conditions.
2. Styling & responsiveness	Consistent UI; effective CSS and dynamic styles; appropriate responsive layouts.	Existing CSS and colors; state-driven profile previews; selected classes; mobile/tablet fixes; 33 page/viewport checks.
3. Hooks & state management	Correct Hooks, effective state/events; dynamic updates behave as expected.	Immutable reducer, functional feed updates, Context, Effects with cleanup, controlled forms, rollback and duplicate guards.
4. Routing & forms	Correct navigation; controlled inputs; effective validation and submission handling.	BrowserRouter route matching; useSearchParams settings tabs; Back/Forward/reload; account/profile/comment forms; server-error and pending handling.
5. Individual understanding & demonstration	Confident explanation; independent problem solving; significant individual contribution.	Source walkthroughs, timed demo, failure exercises and contribution worksheet. You must practise and demonstrate these personally.

How to use this mapping

For each criterion, show a visible feature, open the relevant source, explain how state/props/events produce the result, and mention a test that verifies it. Do not replace explanation with a list of Hook names.

No automatic 20/20 claim: code and tests can provide technical evidence for criteria 1-4. They cannot establish your personal understanding or contribution. Use the exercises near the end to prepare for criterion 5.

Rubric source: the user-supplied image “WhatsApp Image 2026-10-27 at 11.27.37 AM.jpeg”, titled React UI Project - Final Evaluation Rubric (20 Marks). The rubric is treated as task data, not as instructions from an external authority.

Module V coverage and reading map

Module V topic	Chapter / working project evidence	Page
React JS overview	React overview, JSX and the root tree	5
Getting started	Project architecture and run commands	4
State management	State management and events	10
Event handling	Function types used in the project	33
Conditional rendering	Props, conditional rendering and keys	7
Components	Reusable post components	6
Styling / responsiveness	Styling and dynamic profile previews	8
State in the component tree	State in trees, portals and custom Hooks	15
React Context	React Context and shared session state	14
useEffect	useEffect and browser synchronization	13
Hooks in React	Hook inventory with useful examples	16
React Router	React Router: pages and query state	17
Forms	Reusable controlled fields and forms	18
Deployment and/or testing	Deployment and testing setup	24

Rubric preparation

2 - The exact rubric: evidence for each criterion

28 - Your eight-minute evaluation demonstration

29 - Debugging exercises you should solve yourself

30 - Your contribution and explanation worksheet

34 - Source index: open these files first

Study the whole module, but demonstrate the project through the five rubric criteria. Use the source index to move quickly from a viva question to the corresponding implementation.

START HERE BEFORE EXPLAINING THE CODE

Project architecture and run commands

Practice saying: My Reddit Clone uses React and Vite for the frontend, Express for HTTP/API behavior, and MongoDB through Mongoose for persistence. React Context distributes session data. React Router matches existing page URLs, and settings uses query parameters as routing state. Post cards and settings are reusable JSX components. Some existing screens retain template-backed controllers to preserve behavior.

Layer	Responsibility
Browser / React	Render components, own UI state, handle input and call /api.
Vite	Transform JSX/modules, provide development hot reload, generate dist.
Express / Node	Serve built pages, verify authentication, validate requests and return JSON.
MongoDB / Mongoose	Store users, communities, posts, comments, votes and other records.

PROJECT COMMANDS

```
cd /Users/yazir/Desktop/awt2
npm install
# Terminal 1: local MongoDB
npm run db
# Terminal 2: builds React, then starts Express
npm start
# Optional Terminal 3: Vite development frontend
npm run dev:frontend
```

Run everything from the folder containing package.json. Express serves the built app at localhost:3000. Vite runs at localhost:5173 and proxies /api and /uploads to port 3000. MongoDB runs on 27017. npm run dev starts the frontend and backend watchers together; the database still needs to be available.

Source: package.json, lines 7-21

Source: vite.config.mjs, lines 13-25

Q. Does React connect directly to MongoDB?
A. No. The frontend calls the API. Backend routes and Mongoose handle database access.

Q. Why is this still called hybrid?
A. Some pages use original controller logic with React-rendered templates. The core post components and settings forms now use declarative JSX, state and event props directly.

React overview, JSX and the root tree

React is a JavaScript library for describing interfaces as components. Components receive props and may own state. JSX is transformed into JavaScript element creation. A state change schedules rendering; React compares element descriptions and commits needed DOM changes. ReactDOM is the browser renderer, while Vite is the build tool.

Exact code: frontend/react/main.jsx, lines 1-7

```
1  import React from 'react';
2  import { createRoot } from 'react-dom/client';
3  import App from './App.jsx';
4  import { BrowserRouter } from 'react-router';
5  import { AuthProvider } from './AuthContext.jsx';
6
7  createRoot(document.getElementById('root')).render(<BrowserRouter><AuthProvider><App
     /></AuthProvider></BrowserRouter>);
```

createRoot mounts into the real #root element. BrowserRouter supplies routing context. AuthProvider supplies the session snapshot. App selects a page and creates the shared layout. This is client rendering; the project does not use hydrateRoot or server-rendered React HTML.

Exact code: frontend/react/App.jsx, lines 21-29

```
21 export default function App() {
22     const navigate = useNavigate();
23     useLayoutEffect(() => registerNavigation(navigate), [navigate]);
24     return <Routes>
25         <Route path="/" element={<PageLayout filename="index.html" Page={pages['index.html']}
             />} />
26         {Object.entries(pages).map(([filename, Page]) => <Route key={filename}
             path={`/${filename}`} element={<PageLayout filename={filename} Page={Page} />} />)}
27         <Route path="*" element={<PageLayout filename="page.html" Page={pages['page.html']}
             />} />
28     </Routes>;
29 }
```

Route elements select reusable PageLayout instances with different Page props. The page map uses React.lazy. PageLayout wraps its page in Suspense while a module loads. Suspense does not automatically manage ordinary feed API requests; Feed has explicit loading/error state.

Q. Why must component names begin with capitals?
A. JSX treats lowercase tags as built-in DOM elements. Capitalized names refer to JavaScript component values.

Q. Does a .jsx extension alone make code React?
A. No. What matters is rendering components and using React APIs. Some .js files also export React components or provide controller/helpers.

Q. Is AuthStateManager a React class component?
A. No. It is an ordinary JavaScript store class. AuthProvider adapts it for React consumers.

Theory reference: React: Quick Start

Reusable post components

Exact code: frontend/react/PostContent.jsx, lines 12-19

```
12  export function VoteControls({ post, compact = false }) {
13      const active = post.userVote === 1 ? 'voted-up' : post.userVote === -1 ? 'voted-down' :
          '';
14      return <div className={`${compact ? 'compact-pill' : 'action-pill'} vote-pill
          ${active}`}>
15          <button className="vote-btn vote-up" data-action="upvote" aria-label="Upvote"
            aria-pressed={post.userVote === 1}><PostIcon name="up" /></button>
16          <span className="vote-count">{formatCount(post.score)}</span>
17          <button className="vote-btn vote-down" data-action="downvote" aria-label="Downvote"
            aria-pressed={post.userVote === -1}><PostIcon name="down" /></button>
18      </div>;
19  }
```

VoteControls is reused in full and compact cards. post supplies score and vote direction; compact selects the wrapper class. The same JSX structure drives both variants. aria-pressed reflects state, and the data-action attributes connect buttons to the card's React click handler.

Exact code: frontend/react/PostContent.jsx, lines 21-26

```
21  export function JoinButton({ community, compact = false }) {
22      const { user } = useAuth();
23      const joined = user?.joinedCommunities?.some(c => c.name.toLowerCase() ===
          community.name.toLowerCase());
24      if (joined) return null;
25      return <button className={compact ? 'compact-btn-join' : 'btn-post-join'}
          data-action="join" data-join-community={community.name}>Join</button>;
26  }
```

JoinButton reads session Context. If the community is already joined, it returns null. Otherwise it renders the right full/compact button. This is an actual reusable component with conditional rendering, not an unused syllabus example.

Component	Single responsibility
PostCard	Owns post interaction state, API actions and optimistic rollback.
CardContent / CompactContent	Render the two layouts from props.
PostMenu	Show actions appropriate to login, authorship and moderation state.
PostMedia / MediaItem	Render active media and pause an inactive video with an Effect.
FormSection / FormField	Share controlled form structure, validation/error display and input bindings.

Q. What makes this better than copying two vote blocks?
A. One component defines the markup and semantics once. Both layouts receive the same behavior and fixes through props.

Q. Why keep API calls out of VoteControls?
A. It is a presentation component. PostCard owns the interaction and persistence policy; the child only describes the controls.

Props, conditional rendering and keys

Exact code: frontend/react/PostCard.jsx, lines 105-111

```
105     const compact = view === 'compact';
106     const props = { post, showCommunity: options.showCommunity !== false, detail:
          !!options.detail, menuOpen, index, revealed };
107     return <article ref={element} onClick={action} data-post-id={post.id} aria-busy={voting}
108         className={compact ? 'compact-post-card feed-post rc-clickable' : `post-card
              feed-post ${options.detail ? 'rc-post-detail' : 'rc-clickable'}`}>
109         {compact ? <CompactContent {...props} /> : <CardContent {...props} />}
110     </article>;
111 }
```

PostCard chooses CardContent or CompactContent from the view state and passes one props object. It returns null after removal, and a separate placeholder while hidden. The component owns its local menu/media state; it reports post-data changes upward through onChange.

Exact code: frontend/react/Feed.jsx, lines 103-109

```
103         <div className={`posts-feed ${view === 'compact' ? 'compact-view-active' : ''}`}
              id="postsFeedList">
104             {posts.map(post => <PostCard key={`${view}:${post.id}`} post={post} view={view}
                  showCommunity={showCommunity} onChange={changes => setPosts(current =>
                  current.map(item => item.id === post.id ? { ...item, ...changes } : item))}
                  />)}
105         </div>
106         <div className="rc-feed-status" id="feedStatus">
107             {loading ? <div className="rc-spinner" aria-label="Loading" /> : error ? <div
                  className="rc-empty-state"><p>{error}</p><button className="ui-btn
                  ui-btn-secondary" id="feedRetryBtn" onClick={loadMore}>Try again</button></div>
108                 : !posts.length ? <Markup html={emptyHtml} /> : !hasMore ? <p
                      className="rc-feed-end">You've reached the end</p> : null}
109         </div>
```

Feed owns its list of posts and maps each to PostCard. The callback replaces the changed array item immutably. Keys combine view and id: id identifies the post, while switching layout deliberately remounts the card. Since the parent cache receives changes, a saved post stays saved across that remount.

Q. Are props mutable?
A. Treat them as read-only. The current reducer creates new post objects; it does not modify initialPost or the parent array.

Q. What is lifting state up?
A. Moving shared ownership to a common parent and passing values/callbacks down. Feed keeps the post list, and its onChange callback updates the right cached item.

Q. What does returning null do?
A. That component produces no DOM output. A component can still be mounted while returning null; its Effects are not automatically removed until it unmounts.

Q. Does React pass key to a child as a prop?
A. No. React uses key internally for identity. Pass id separately if the child needs it.

Theory reference: React: Conditional Rendering

Styling and dynamic profile previews

The application uses existing global CSS, custom properties, media queries, state-dependent className values and some inline style objects. There is no Tailwind or CSS Modules setup. React controls the values that change; CSS supplies the layout and visual rules.

Exact code: frontend/react/SettingsPage.jsx, lines 68-80

```
68              <div className="rc-avatar-editor">
69                  <div className="rc-avatar-preview" id="avatarPreview">
70                      <div className="rc-avatar-preview-banner no-invert" id="bannerPreview"
                    style={{ backgroundColor: form.values.bannerColor }} />
71                      <div className="rc-avatar-preview-img" id="avatarPreviewImg"><Markup
                    html={userAvatar({ avatarColor: form.values.avatarColor }, 72)} /></div>
72                  </div>
73                  <div>
74                      <span className="ui-field-label">Avatar color</span>
75                      <div className="rc-swatches" id="avatarSwatches">
76                          {SWATCHES.map(color => <button key={color} type="button"
                    className={`rc-swatch no-invert ${color === form.values.avatarColor ?
                    'selected' : ''}`} style={{ background: color }} data-color={color}
                    aria-label={`Avatar color ${color}`} aria-pressed={color ===
                    form.values.avatarColor} onClick={() => form.change('avatarColor', color)}
                    />)}
77                          <input type="color" className="ui-color" id="avatarColorInput"
                    value={form.values.avatarColor} aria-label="Custom avatar color"
                    onChange={e => form.change('avatarColor', e.target.value.toUpperCase())} />
78                      </div>
79                      <label className="ui-field-label" htmlFor="bannerColorInput">Banner
                    color</label>
80                      <input type="color" className="ui-color" id="bannerColorInput"
                    value={form.values.bannerColor} onChange={e => form.change('bannerColor',
                    e.target.value.toUpperCase())} />
```

Profile form state owns avatarColor and bannerColor. The banner reads backgroundColor from state. Swatch buttons use the same color as an inline style, a selected class and aria-pressed. The native color picker is controlled through value/onChange. Clicking either input updates the preview before any server save.

Banner color event: frontend/react/SettingsPage.jsx, lines 80-80

Existing profile layout styles: frontend/css/pages.css, lines 791-805

Q. className versus style?
A. className selects reusable CSS rules. style accepts a JavaScript object for values that depend on data, such as the selected profile color.

Q. Why use camelCase in inline styles?
A. React style props are JavaScript object keys, for example backgroundColor rather than background-color.

Q. Does an unsaved color change update the server?
A. No. It changes controlled local state and the preview. Submitting the profile form sends the values to /users/me.

Q. How is dark mode implemented?
A. It remains a document class and localStorage preference. The React preference toggle controls the switch; existing CSS filters and media exclusions produce the appearance.

Also inspect: frontend/js/display-mode.js, lines 11-20

Responsive layouts and the bugs fixed

Exact code: frontend/css/responsive.css, lines 28-38

```
28  @media (max-width: 767px) {
29      .main-layout,
30      .main-layout.sidebar-collapsed,
31      .main-layout.no-right-col,
32      .main-layout.no-right-col.sidebar-collapsed {
33          grid-template-columns: minmax(0, 1fr);
34          padding: var(--space-md) var(--space-sm);
35          gap: 0;
36      }
37
38      #sidebar-container {
```

The previous mobile rule targeted only .main-layout. More-specific desktop modifiers could keep extra grid columns on full-width pages. The selectors now cover ordinary, collapsed-sidebar and no-right-column states. minmax(0, 1fr) permits the remaining column to shrink to the viewport.

Exact code: frontend/components/post-create/post-create.css, lines 477-483

```
477  .toolbar-left {
478      display: flex;
479      flex-wrap: wrap;
480      min-width: 0;
481      align-items: center;
482      gap: 4px;
483  }
```

The rich-text toolbar now wraps its buttons instead of extending beyond the editor. This fixes a real 390px/768px overflow without hiding controls. The tablet press asset grid uses two columns so cards fit the available content region.

Tablet asset grid: frontend/css/pages.css, lines 989-991

How it was verified

Browser tests visit all 11 page entries at 390, 768 and 1440 pixels and assert that neither the document nor the feed requires horizontal scrolling. A separate check verifies that a saved collapsed-sidebar preference does not squeeze phone/tablet content. Existing desktop and mobile-home baseline screenshots remain unchanged.

Q. Why not just add overflow-x: hidden?
A. That could conceal inaccessible controls. The fix changes the grid and wrapping so the actual content fits.

Q. What is CSS specificity?
A. It helps decide which matching rule wins. The earlier .main-layout.no-right-col selector was more specific than a mobile .main-layout rule.

Q. Does checking three widths prove all responsiveness?
A. No. It covers representative phone/tablet/desktop widths. Unusual content, zoom levels and other widths may need further checks.

State management and events

State is component memory that survives renders. useState returns a value and a setter. Calling the setter schedules a render; it does not immediately replace the variable in the current handler. A ref stores mutable coordination data without scheduling a render. Props provide inputs from a parent.

Exact code: frontend/react/Feed.jsx, lines 31-43

```
31      const { user } = useAuth();
32      const params = new URLSearchParams(window.location.search);
33      const [sort, setSort] = useState(() => sorts.includes(params.get('sort')) ?
          params.get('sort') : defaultSort);
34      const [time, setTime] = useState(() => TIME_LABELS[params.get('t')] ? params.get('t') :
          'day');
35      const [view, setView] = useState(() => readLocal('rc_view', null) ||
          user?.prefs?.defaultView || 'card');
36      const [posts, setPosts] = useState([]);
37      const [loading, setLoading] = useState(true);
38      const [error, setError] = useState('');
39      const [hasMore, setHasMore] = useState(true);
40      const [menu, setMenu] = useState(null);
41      const [revision, setRevision] = useState(0);
42      const sentinel = useRef(null);
43      const request = useRef({ generation: 0, page: 0, busy: false, more: true });
```

Sort, time, view, fetched posts and visible request/menu status are state because they affect UI. The initializers read the URL and saved preference only when this Feed instance mounts. sentinel and request are refs used to connect browser/API behavior to the component.

Exact code: frontend/react/forms/useForm.js, lines 4-12

```
4   export function useForm({ initialValues, validate, onSubmit, resetOnSuccess = false }) {
5       const [values, setValues] = useState(initialValues);
6       const [errors, setErrors] = useState({});
7       const [pending, setPending] = useState(false);
8       const inFlight = useRef(false);
9       function change(name, value) {
10          setValues(current => ({ ...current, [name]: value }));
11          setErrors(current => ({ ...current, [name]: undefined, form: undefined }));
12      }
```

The custom form Hook owns values, errors and pending state. Its change function creates a new values object using a functional updater. It clears the changed field error and any server error so the user can correct and resubmit.

Q. Why use the functional setter here?
A. A field update depends on the previous values object. The updater merges into the latest state without discarding other fields.

Q. Is event handling the same as state management?
A. No. An event is the trigger; the handler decides how to update state or call an API. React then renders the result of the new state.

Q. What events are used?
A. Post cards and tabs use onClick. Controlled fields use onChange. Forms use onSubmit. CommentComposer also handles onKeyDown for Ctrl/Cmd + Enter.

Q. Why call preventDefault in a submit handler?
A. It stops the browser's normal form navigation/reload so the app can validate and submit asynchronously.

Theory reference: React: useState

An immutable reducer for post data

Exact code: frontend/react/postState.mjs, lines 1-8

```
1  /** Pure updates: the original post/props are never mutated. */
2  export function postReducer(post, action) {
3      switch (action.type) {
4          case 'vote': return { ...post, score: post.score + action.direction - post.userVote,
             userVote: action.direction };
5          case 'patch': return { ...post, ...action.changes };
6          default: return post;
7      }
8  }
```

The reducer receives the previous post and an action, then returns the next post. vote computes the net score change when moving from one direction to another. patch merges confirmed fields or rollback fields. The spread expressions create a new object, preserving the input snapshot.

Exact code: frontend/react/PostCard.jsx, lines 12-24

```
12 export default function PostCard({ post: initialPost, ...options }) {
13     const { isLoggedIn } = useAuth();
14     const element = useRef(null);
15     const [post, dispatch] = useReducer(postReducer, initialPost);
16     const votePending = useRef(false);
17     const [menuOpen, setMenuOpen] = useState(false);
18     const [hidden, setHidden] = useState(false);
19     const [removed, setRemoved] = useState(false);
20     const [expanded, setExpanded] = useState(false);
21     const [index, setIndex] = useState(0);
22     const [revealed, setRevealed] = useState(false);
23     const [voting, setVoting] = useState(false);
24     const view = expanded ? 'card' : options.view;
```

useReducer now holds actual post data. The previous implementation used a counter only to force a render after mutating props; that workaround has been removed. Local menu, hidden/removed, expanded and media state remain separate because they describe the current card UI.

Transition	Score change
Neutral (0) to upvote (+1)	+1
Upvote (+1) to neutral (0)	-1
Upvote (+1) to downvote (-1)	-2
Downvote (-1) to upvote (+1)	+2

Q. Why is +1 to -1 a change of -2?
A. The previous +1 must first be removed, then a -1 contribution added. The reducer subtracts the old direction from the new direction.

Q. Can a reducer make an API request?
A. A reducer should remain pure: same input/action gives the same output without side effects. PostCard's async handler performs the request outside the reducer.

Q. What proves the old object is not mutated?
A. The unit test freezes the original object and checks every old/new vote combination. It also verifies rollback preserves an independent saved flag.

Theory reference: React: Updating Objects in State

Async actions, optimistic UI and rollback

Exact code: frontend/react/PostCard.jsx, lines 34-45

```
34      async function vote(dir) {
35          if (!isLoggedIn) { requireLogin(); return; }
36          if (votePending.current) return;
37          votePending.current = true;
38          const next = post.userVote === dir ? 0 : dir;
39          const previous = { score: post.score, userVote: post.userVote };
40          dispatch({ type: 'vote', direction: next });
41          options.onChange?.({ score: post.score + next - post.userVote, userVote: next });
42          setVoting(true);
43          try { update(await api.post(`/posts/${post.id}/vote`, { dir: next })); }
44          catch (err) { update(previous); showToast(err.message); }
45          finally { votePending.current = false; setVoting(false); }
```

The handler first checks login and a synchronous pending ref. It computes whether the click selects a direction or toggles it to neutral, remembers the previous score/vote, dispatches an optimistic update and tells the parent cache. The API result confirms the fields. An error restores only the vote fields and shows a toast. finally releases the guard and pending UI.

Exact code: frontend/react/SettingsPage.jsx, lines 101-111

```
101     async function save(patch, label) {
102         const previous = prefs;
103         setPrefs(current => ({ ...current, ...patch })); setPending(true);
104         try {
105             const { user: updated } = await api.patch('/users/me', { prefs: patch });
106             authState.setUser(updated); setPrefs(updated.prefs);
107             if (patch.defaultView) writeLocal('rc_view', patch.defaultView);
108             showToast(`${label} saved`);
109         } catch (error) { setPrefs(previous); showToast(error.message); }
110         finally { setPending(false); }
111     }
```

Preferences use the same broad pattern: save the old snapshot, show the selected value immediately, then confirm it from the API or restore it on error. The default-view localStorage value is written only after a successful server response, so failure does not leave the browser preference inconsistent with the screen.

Q. What is optimistic UI?
A. Displaying the expected result before the server responds. It feels responsive, but requires a defined rollback when the request fails.

Q. Why use both a ref and a state flag for voting?
A. The ref blocks duplicate requests immediately. State controls visible rendering and aria-busy. Ref changes alone would not update the UI.

Q. Why restore only score and userVote?
A. Another independent update, such as saving a post, should survive a failed vote. A whole stale-object replacement could erase that change.

useEffect and browser synchronization

An Effect synchronizes a committed component with external work such as fetches, observers, listeners or timers. Dependencies identify the reactive values it uses. Cleanup runs before the next relevant setup and when the component unmounts.

Exact code: frontend/react/Feed.jsx, lines 64-80

```
64      useEffect(() => {
65          request.current = { page: 0, busy: false, more: true };
66          setPosts([]); setHasMore(true); loadMore();
67          return () => { request.current = { page: 0, busy: true, more: false }; };
68      }, [loadMore, revision]);
69      useEffect(() => {
70          const observer = new IntersectionObserver(entries => { if (entries.some(e =>
              e.isIntersecting)) loadMore(); }, {
71              root: sentinel.current?.closest('#feed-container') || null, rootMargin: '600px
                0px'
72          });
73          if (sentinel.current) observer.observe(sentinel.current);
74          return () => observer.disconnect();
75      }, [loadMore, revision]);
76      useEffect(() => {
77          const close = () => setMenu(null);
78          document.addEventListener('click', close);
79          return () => document.removeEventListener('click', close);
80      }, []);
```

The first Effect resets pagination and starts loading when the callback or revision changes. The second watches the sentinel using IntersectionObserver and disconnects on cleanup. The third installs and removes the document click handler used to close menus.

Exact code: frontend/react/PostContent.jsx, lines 46-51

```
46  function MediaItem({ media, title, active }) {
47      const video = useRef(null);
48      useEffect(() => { if (!active) video.current?.pause(); }, [active]);
49      const props = { className: `post-image rc-media-item ${active ? '' : 'hidden'}`, src:
          media.url };
50      return media.type === 'video' ? <video ref={video} {...props} controls preload="metadata"
          /> : <img {...props} alt={title} loading="lazy" />;
51  }
```

MediaItem receives active as a prop. An Effect pauses its video when that item becomes inactive. React now controls media visibility through className instead of manually repainting the card DOM in a layout Effect.

Q. Why not perform fetch in the render body?
A. A render can occur repeatedly and should describe UI. The Effect ties the request lifecycle to committed dependencies.

Q. Why is the Effect callback not async?
A. An async function returns a Promise. React expects either no return value or a cleanup function. The setup calls an async helper instead.

Q. How does the feed reject stale responses?
A. Each loading cycle stores a new request marker. A completed response updates posts only when its marker is still current. It ignores stale results; it does not abort the HTTP request.

Theory reference: React: useEffect

React Context and shared session state

Exact code: frontend/react/AuthContext.jsx, lines 1-19

```jsx
import React, { createContext, useContext, useSyncExternalStore } from 'react';
import { authState } from '../js/auth-state.js';

const AuthContext = createContext(null);
const subscribe = listener => authState.subscribe(listener);
const getSnapshot = () => authState.getState();

// Template regions have their own React roots. Each provider subscribes to the
// same session store so every tree receives the same current session snapshot.
export function AuthProvider({ children }) {
    const session = useSyncExternalStore(subscribe, getSnapshot);
    return <AuthContext.Provider value={session}>{children}</AuthContext.Provider>;
}

export function useAuth() {
    const session = useContext(AuthContext);
    if (!session) throw new Error('useAuth must be used inside AuthProvider');
    return session;
}
```

createContext creates the channel. AuthProvider reads the existing session store with useSyncExternalStore and supplies its snapshot. useAuth is a custom Hook wrapping useContext with a clear missing-provider error. A new snapshot causes consumers to update.

Where this is actually used

Feed uses the user's default view preference. PostCard checks login before voting. JoinButton checks membership. PostMenu checks login and moderation display preference. Settings renders a login prompt or controlled account/profile/preferences components.

Also inspect: frontend/react/PostContent.jsx, lines 28-30

Also inspect: frontend/react/SettingsPage.jsx, lines 127-131

Q. Why not put every value in Context?
A. Local form text, menu state and loading indicators are owned where they are used. Session data is shared because multiple independent features need it.

Q. Does Context secure protected APIs?
A. No. It describes client session state. Express must verify cookies and enforce authorization for every protected operation.

Q. Why immutable session snapshots?
A. useSyncExternalStore compares snapshots by identity. authState.setUser returns a new state object after a real update; getState otherwise returns the cached object.

Membership snapshot update: frontend/js/auth-state.js, lines 60-67

Theory reference: React: useSyncExternalStore

State in trees, portals and custom Hooks

SIMPLIFIED ACTUAL COMPONENT TREE

```
BrowserRouter -> AuthProvider -> App -> PageLayout
                                     -> SettingsPage
                                        -> portal: Settings
                                           -> ProfileSettings
                                              -> FormSection
                                                 -> FormField
Regional root: AuthProvider -> Feed -> PostCard -> VoteControls
```

A portal changes the DOM destination while preserving React ancestry. Settings can therefore render into #feed-container and still read Router/Auth Context from the main root. By contrast, the retained template renderer creates separate React roots; each needs its own AuthProvider.

Exact code: frontend/react/SettingsPage.jsx, lines 141-147

```
141  export default function SettingsPage() {
142      const [ready, setReady] = useState(false);
143      const initialize = useCallback(async () => { await initShell(); setReady(true); }, []);
144      usePage(initialize);
145      // A portal preserves the document root's Router and Auth Context ancestry.
146      return ready ? createPortal(<Settings />, document.getElementById('feed-container')) :
           null;
147  }
```

The page stores readiness, memoizes its initializer and passes it to usePage. After the shell creates the destination slot and loads the session, it returns the portal. The initializer identity stays stable so ordinary rerenders do not restart the whole page.

Custom Hook	Reusable behavior
useAuth	Read session Context and enforce the provider requirement.
usePage	Initialize a page after the commit; handle startup failure and dispose regional roots.
useForm	Own controlled values, validate, guard submission and expose errors/pending state.

Q. Does a custom Hook share one state instance across all calls?
A. No. It shares logic. Each useForm call owns its own values/errors/pending state. Shared data requires Context or an external store.

Q. Does DOM nesting make two roots share Context?
A. No. Context follows React ancestry, not arbitrary DOM containment. A portal preserves that ancestry; a new root starts a separate tree.

Q. Why are Hook names prefixed with use?
A. The naming convention identifies React-aware functions and lets lint/tooling apply Hook rules. Ordinary helpers such as validateAccount do not need the prefix.

Hook inventory with useful examples

Hook	Current working use
useState	Feed controls/status; card menus/media; controlled form values, errors and pending state.
useReducer	PostCard owns post data using the pure postReducer. It is no longer a refresh counter.
useEffect	Feed requests/observer/listener; MediaItem pauses inactive video; comment focus; toast timers.
useContext	useAuth reads the session provided by AuthProvider.
useRef	DOM focus/observer references and immediate duplicate-request guards.
useMemo	Markup caches parsed React elements when its HTML string has not changed.
useCallback	Feed stabilizes loadMore; SettingsPage stabilizes its page initializer.
useLayoutEffect	App registers the Router navigation bridge before page initialization can use it.
useSyncExternalStore	AuthProvider and Toasts subscribe to external stores with stable snapshots.
useAuth / usePage / useForm	Project-defined custom Hooks for session, page startup and forms.
useNavigate / useSearchParams	React Router Hooks for navigation and URL-driven settings tabs.

Exact code: frontend/react/render.jsx, lines 37-39

```
37  export function Markup({ html }) {
38      return useMemo(() => parse(String(html || ''), options), [html]);
39  }
```

useMemo stores a calculated value; useCallback stores a function reference. They are optimizations, not persistence or substitutes for correct dependencies. Markup parses trusted/escaped input; it is not an HTML sanitizer.

Q. Must every React Hook be used for this rubric?
A. No. The rubric explicitly names useState/useEffect and correct behavior. Every Hook used here has a purpose; unused advanced Hooks should not be added just to increase a count.

Q. What are the main Hook rules?
A. Call state/effect/context Hooks at the top level of components or custom Hooks, not in conditions, loops or ordinary event callbacks.

Q. Is createContext a Hook?
A. No. It creates a Context object. useContext is the Hook that reads it.

React Router: pages and query state

BrowserRouter provides normal browser-location routing. Routes chooses a Route element based on the path. The project keeps its existing .html URLs and uses lazy page components. Settings uses the query string as its tab state, so the URL and rendered form stay connected.

Exact code: frontend/react/SettingsPage.jsx, lines 127-139

```
127 function Settings() {
128     const { isLoggedIn } = useAuth();
129     const [params, setParams] = useSearchParams();
130     const tab = Object.hasOwn(TABS, params.get('tab')) ? params.get('tab') : 'account';
131     if (!isLoggedIn) return <div className="rc-empty-state"><h3>Log in to manage your
          settings</h3><button className="ui-btn ui-btn-primary" id="settingsLoginBtn"
          onClick={() => openAuthModal('Log In')}>Log In</button></div>;
132     return <div className="rc-settings-page">
133         <h1 className="rc-view-title">Settings</h1>
134         <nav className="rc-tabs" role="tablist" aria-label="Settings">
135             {Object.entries(TABS).map(([key, label]) => <button key={key} className={`rc-tab
              ${key === tab ? 'active' : ''}`} data-tab={key} role="tab" aria-selected={key
              === tab} onClick={() => setParams({ tab: key })}>{label}</button>)}
136         </nav>
137         <div id="settingsContent">{tab === 'profile' ? <ProfileSettings /> : tab ===
            'preferences' ? <PreferencesSettings /> : <AccountSettings />}</div>
138     </div>;
139 }
```

useSearchParams returns the current parameters and a setter that navigates. The selected tab is derived from the URL rather than maintained in a second local variable. Invalid tab names fall back to Account using an own-property check. Each button updates the query; the matching component renders conditionally.

What is verified

The browser test opens Profile, changes tabs, presses Back/Forward without a document reload, reloads the selected tab, and checks an invalid tab parameter. It also verifies active-tab semantics through aria-selected.

Page route matching: frontend/react/App.jsx, lines 21-29

Navigation test: tests/rubric.spec.mjs, lines 55-73

Q. Path parameter versus query parameter?
A. /posts/:id would define a path parameter. This app uses /post.html?id=... and /settings.html?tab=... as query parameters. It does not use useParams.

Q. Does every link now use client-side SPA navigation?
A. No. Settings tabs and existing query controllers navigate in place. Cross-page anchors still load documents so retained controllers release their document listeners/timers.

Q. What serves a directly opened route?
A. Express serves each known HTML entry from dist. Client-side Route definitions alone do not configure HTTP server fallbacks.

Theory reference: React Router: useSearchParams

Reusable controlled fields and forms

A controlled input receives its current value from React state and updates that state through an event handler. Checkboxes use checked and event.target.checked. Uncontrolled inputs keep the live value in the DOM. This project uses controlled settings/comment forms while retaining an uncontrolled rich-text editor.

Exact code: frontend/react/forms/FormSection.jsx, lines 4-12

```
4  export function FormField({ form, id, name, label, multiline = false, ...props }) {
5      const Input = multiline ? 'textarea' : 'input';
6      return <>
7          <label className="ui-field-label" htmlFor={id}>{label}</label>
8          <Input {...props} id={id} name={name} className={`ui-input${multiline ? '
             ui-textarea' : ''}`}
9              value={form.values[name]} aria-invalid={!!form.errors[name]}
10             onChange={event => form.change(name, event.target.value)} />
11     </>;
12 }
```

FormField chooses input or textarea from multiline, associates the label with htmlFor/id, reads the field value from the shared form object and routes edits through form.change. aria-invalid reflects field validation state. The same component is reused for email, passwords, display name and bio.

Exact code: frontend/react/forms/FormSection.jsx, lines 14-24

```
14 export default function FormSection({ id, title, description, submitLabel, danger = false,
     children, ...options }) {
15     const form = useForm(options);
16     const error = Object.values(form.errors).filter(Boolean).join('. ');
17     return <form className="rc-settings-section" id={id} noValidate onSubmit={form.submit}
         aria-busy={form.pending}>
18         <h2>{title}</h2>
19         {description && <p className="rc-muted-text">{description}</p>}
20         {children(form)}
21         <p className={`ui-error${error ? '' : ' hidden'}`} data-error=""
             role="alert">{error}</p>
22         <div className="ui-dialog-actions"><button type="submit" className={`ui-btn ${danger
             ? 'ui-btn-danger' : 'ui-btn-primary'}`}
             disabled={form.pending}>{submitLabel}</button></div>
23     </form>;
24 }
```

FormSection composes the useForm Hook with consistent title, description, fields, error summary and submit button. It passes the form API to children as a render callback. noValidate disables browser popups because the explicit validator supplies consistent inline messages.

Q. What happens if value is provided without onChange?
A. The displayed value cannot follow typing unless state is updated, so the field behaves as read-only.

Q. Why pass children as a function?
A. The parent can choose fields and preview content while receiving the shared form state/handlers. This separates reusable form mechanics from specific form content.

Q. Why disable submission while pending?
A. It communicates progress and prevents repeated user actions. The Hook also uses a ref guard to reject duplicate requests synchronously.

Theory reference: React: Controlled Inputs

Validation and asynchronous submission

Exact code: frontend/react/forms/validation.mjs, lines 1-13

```
export function validateAccount(kind, values) {
    const errors = {};
    if (kind === 'email') {
        if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(values.email.trim())) errors.email = 'Enter a valid email address';
        if (!values.password) errors.password = 'Enter your current password';
    } else if (kind === 'password') {
        if (!values.currentPassword) errors.currentPassword = 'Enter your current password';
        if (values.newPassword.length < 8) errors.newPassword = 'Use at least 8 characters for the new password';
        if (values.newPassword !== values.confirmPassword) errors.confirmPassword = "New passwords don't match";
    } else if (!values.password) errors.password = 'Enter your password to confirm';
    return errors;
}

```

The validator returns a field-to-message object. Email needs a plausible address and current password. Password change requires the current password, at least eight characters and matching confirmation. Account deletion requires a password before asking for confirmation. The profile validator separately checks display-name/bio limits.

Exact code: frontend/react/forms/useForm.js, lines 13-31

```
    async function submit(event) {
        event.preventDefault();
        if (inFlight.current) return;
        const nextErrors = validate(values);
        setErrors(nextErrors);
        if (Object.keys(nextErrors).length) {
            event.currentTarget.elements.namedItem(Object.keys(nextErrors)[0])?.focus();
            return;
        }
        inFlight.current = true;
        setPending(true);
        try {
            const result = await onSubmit(values);
            if (resetOnSuccess && result !== false) setValues(initialValues);
        } catch (error) {
            setErrors({ form: error.message });
        } finally {
            inFlight.current = false;
            setPending(false);
```

submit prevents normal navigation, checks the immediate guard, runs validation and focuses the first invalid field. Valid input starts the request and disables the button. A successful resettable form clears its fields; a thrown server error becomes the form error. finally releases pending state for retry.

Q. Does client validation replace server validation?
A. No. A caller can bypass the UI. Server routes must still validate and authorize requests.

Q. Why is the server error kept separate from field errors?
A. A rejected password or unavailable service may not be a formatting problem. The form-level message explains the failed submission while preserving typed values.

Q. Why can onSubmit return false?
A. For example, cancelling the account-deletion confirmation should not be treated as a successful submission/reset.

Validation and request test: tests/rubric.spec.mjs, lines 24-28

Comments, replies and edits

Exact code: frontend/react/CommentComposer.jsx, lines 33-47

```
33      return <form className="rc-comment-composer" onSubmit={submit}>
34          <textarea ref={textarea} className="rc-comment-textarea" placeholder={placeholder}
              maxLength={10000} rows={3}
35              value={value} onChange={event => setValue(event.target.value)} onKeyDown={event
                => {
36                  if (event.key === 'Enter' && (event.ctrlKey || event.metaKey)) {
37                      event.preventDefault();
38                      event.currentTarget.form.requestSubmit();
39                  }
40              }} />
41          <div className="rc-composer-actions">
42              <span className="rc-muted-text rc-composer-hint">Ctrl + Enter to submit</span>
43              {onCancel && <button type="button" className="ui-btn ui-btn-secondary"
                data-cancel="" onClick={onCancel}>Cancel</button>}
44              <button type="submit" className="ui-btn ui-btn-primary" disabled={submitting ||
                !value.trim()}>{submitLabel}</button>
45          </div>
46      </form>;
47  }
```

CommentComposer stores its text with useState and receives onSubmit, onCancel, initialValue and labels as props. The parent uses the same component for a new comment, reply or edit. Ctrl/Cmd + Enter uses requestSubmit, so keyboard and button submissions follow the same validation and duplicate-prevention path.

Parent persistence callback: frontend/js/pages/post-page.js, lines 154-163

Scenario	Observed behavior
Whitespace-only input	Submit is disabled and the handler also rejects empty trimmed text.
Pending request	Duplicate keyboard/click submissions are blocked.
Server failure	Toast shows the error, text is retained, retry becomes available.
Successful new comment	API response updates the comment tree; text resets.
Cancel reply/edit	The form closes and the original comment remains unchanged.

Q. Why have both disabled UI and a handler guard?
A. UI state improves feedback, but the handler must also protect against rapid repeated events or programmatic submissions.

Q. How does the reusable form know which endpoint to call?
A. It does not choose an endpoint. Its parent supplies the onSubmit callback, allowing comment, reply and edit persistence to differ.

Q. Why does initialValue not overwrite typing on every render?
A. useState uses it to initialize the mounted instance. Fresh keyed form mounts initialize new edits; ordinary rerenders preserve current typing.

Trace a feature from browser to database

Stage	Your project behavior
1. Controlled input	CommentComposer onChange updates value. UI reflects the new state.
2. Submission	The React handler prevents reload, trims, guards duplicates and awaits the parent callback.
3. Parent logic	post-page.js sends POST /api/posts/:id/comments; replies include parentId.
4. HTTP wrapper	api.js serializes JSON, includes the same-origin session cookie and throws an ApiError for failed responses.
5. Backend checks	Express verifies authentication, post/community access, body length and the parent comment relationship.
6. Persistence	Mongoose creates the comment and vote, updates count/ranks and may create a notification.
7. Success	The parent updates its comment data and renders the tree; the new-comment form clears.
8. Failure	The error becomes a toast, text is retained and pending state is released.

Exact code: server/routes/posts.js, lines 233-248

```
233     const body = String(req.body.body || '').trim();
234     if (!body) throw new HttpError(400, 'Comment cannot be empty');
235     if (body.length > 10000) throw new HttpError(400, 'Comments can be at most 10,000
          characters');
236
237     let parent = null;
238     if (req.body.parentId) {
239         assertId(req.body.parentId, 'Comment');
240         parent = await Comment.findOne({ _id: req.body.parentId, post: post._id });
241         if (!parent) throw new HttpError(404, 'The comment you replied to no longer exists');
242     }
243
244     const comment = await Comment.create({ post: post._id, author: req.user._id, parent:
          parent?._id || null, body });
245     await Vote.create({ user: req.user._id, targetType: 'Comment', target: comment._id, dir:
          1 });
246
247     post.commentCount += 1;
248     applyRanks(post);
```

Q. Which parts are React-specific?
A. The component tree, Hooks, controlled state and event props. HTTP, Express routing and Mongoose persistence are separate technologies used by the feature.

Q. What is the trust boundary?
A. Client UI and Context cannot authorize an operation. The backend must check the real session and validate data before writing.

Q. Why validate parentId against the same post?
A. A reply must belong to a valid comment on that post. Otherwise the tree could contain an invalid cross-post relation.

Every page and its source

Page	Implementation to explain
Home / index.html	frontend/js/app.js initializes the shell and query view. frontend/js/router.js selects Home, Popular, News, All, Explore or a custom feed. Feed.jsx owns sorting, pagination and post rendering.
Post / post.html	frontend/js/pages/post-page.js loads a post, mounts PostCard, builds the threaded comment tree and supplies callbacks to CommentComposer for comments/replies/edits.
Community / community.html	frontend/js/pages/community-page.js loads community data, mounts a filtered Feed and handles joining, settings and moderation dialogs.
Communities / communities.html	frontend/js/pages/communities-page.js renders community management/discovery and opens the creation wizard.
Profile / user.html	frontend/js/pages/user-page.js selects post/comment/saved/hidden/vote/achievement tabs. Reusable Feed/PostCard components render post lists.
Search / search.html	frontend/js/pages/search-page.js selects posts, communities, comments or people according to search parameters.

Exact source entry points

Also inspect: frontend/js/app.js, lines 6-12

Also inspect: frontend/js/pages/post-page.js, lines 89-101

Also inspect: frontend/js/pages/community-page.js, lines 103-113

Also inspect: frontend/js/pages/user-page.js, lines 121-130

Also inspect: frontend/js/pages/search-page.js, lines 51-58

Q. Are all page controllers themselves React components?
A. No. They contain ordinary functions for loading and interaction. The exported Page component invokes usePage, while controllers mount React views into existing layout slots.

Q. How do comments become nested?
A. buildTree groups comments by their parent id; commentHtml traverses that grouping. The new/reply/edit forms are React components, while the tree rendering retains its existing controller.

Every page and its source

Page	Implementation to explain
Submit / submit.html	frontend/js/submit-app.js loads the post editor. frontend/components/post-create/post-create.js manages rich text, link/media, tags, drafts and publishing. The toolbar now wraps correctly on narrow screens.
Settings / settings.html	frontend/js/pages/settings-page.js re-exports SettingsPage.jsx. AccountSettings, ProfileSettings and PreferencesSettings are declarative React. FormSection/FormField/useForm share input behavior.
Careers / careers.html	frontend/js/pages/careers-page.js handles listings, filters, job detail, applications and query navigation.
Press / press.html	frontend/js/pages/press-page.js handles releases, statistics, brand assets and media inquiries. The asset grid now uses two tablet columns.
Information / page.html	frontend/js/pages/info-page.js selects an information page from the p query parameter. Some external Reddit product links intentionally lead to informational content.

Shared infrastructure

shell.js creates the header/sidebar/chat/modal regions and loads the session. api.js wraps fetch and distinguishes JSON from FormData. auth-state.js holds the shared session. render.jsx supports the retained templates and root disposal. interactions.js contains existing authentication dialogs. Server routes and models remain separate from frontend rendering.

Also inspect: frontend/js/shell.js, lines 16-21

Also inspect: frontend/js/submit-app.js, lines 8-15

Also inspect: frontend/js/pages/careers-page.js, lines 28-33

Also inspect: frontend/js/pages/press-page.js, lines 32-37

Q. Is the rich-text editor controlled?
A. No. It remains an uncontrolled contenteditable region to preserve browser selection/paste/toolbar behavior. Controlled React forms are demonstrated elsewhere with actual user workflows.

Q. Is every external feature implemented?
A. No. Google/Apple/Phone sign-in needs provider integration. Some external product links are informational. Explain the project's actual scope.

Deployment and testing setup

Exact code: vite.config.mjs, lines 13-25

```
13      server: {
14          host: '127.0.0.1',
15          port: 5173,
16          proxy: { '/api': 'http://127.0.0.1:3000', '/uploads': 'http://127.0.0.1:3000' }
17      },
18      build: {
19          outDir: '../dist',
20          emptyOutDir: true,
21          assetsDir: 'bundles',
22          rollupOptions: {
23              input: Object.fromEntries(readdirSync('frontend').filter(f =>
                  f.endsWith('.html')).map(f => [f.replace('.html', ''), resolve('frontend',
                  f)]))
24          }
25      }
```

Vite runs on port 5173 with /api and /uploads proxied to Express. Its build writes dist and bundles all HTML entries. npm start runs the prestart build and then server/server.js. A build alone does not start the API or MongoDB.

VALIDATION COMMANDS

```
# From the project root; MongoDB running for browser tests
npm run build
npm run test:unit
npx playwright install chromium
npm run test:e2e
npm run test:visual
```

The browser suite uses port 3002 and the separate reddit_clone_react_test database. The visual runner needs local baseline screenshots and recorded responses in .data/react-parity. Those baselines are not included in a fresh clone. The normal application continues to use port 3000 and its configured database.

Isolated test configuration: playwright.config.mjs, lines 3-15

Q. Can dist alone provide the whole application on static hosting?
A. No. Accounts, posts, comments and uploads require a reachable Express API, database and appropriate cookie/storage configuration.

Q. What is required for production?
A. Configure a real database, JWT secret, SMTP for email-code flows, HTTPS and persistent upload/private-file storage. This review verifies build/serve capability; it does not establish a public hosted deployment.

Q. Why keep tests separate from the normal database?
A. Browser tests create and edit records. Isolation prevents those test flows from changing the normal project dataset.

Theory reference: Vite: Deploying a Static Site

Unit tests: prove state and validation rules

Exact code: tests/state.test.mjs, lines 6-17

```
6  test('all vote transitions preserve the original snapshot and use net score changes', () => {
7      for (const oldVote of [-1, 0, 1]) {
8          for (const nextVote of [-1, 0, 1]) {
9              const original = Object.freeze({ id: 'post', score: 10, userVote: oldVote, saved:
                 false });
10             const next = postReducer(original, { type: 'vote', direction: nextVote });
11             assert.equal(next.userVote, nextVote);
12             assert.equal(next.score - nextVote, original.score - oldVote);
13             assert.equal(original.score, 10);
14             assert.notEqual(next, original);
15         }
16     }
17 });
```

This test explores all nine old/new vote combinations. Freezing the original object makes accidental mutation fail. The invariant checks that the score excluding the user's own contribution stays unchanged; it also checks that the result is a different object.

Exact code: tests/state.test.mjs, lines 19-26

```
19 test('rolling back a vote preserves an independent save operation', () => {
20     const original = Object.freeze({ score: 10, userVote: 0, saved: false });
21     const voted = postReducer(original, { type: 'vote', direction: 1 });
22     const saved = postReducer(voted, { type: 'patch', changes: { saved: true } });
23     const rolledBack = postReducer(saved, { type: 'patch', changes: { score: original.score,
         userVote: original.userVote } });
24     assert.deepEqual(rolledBack, { score: 10, userVote: 0, saved: true });
25     assert.equal(original.saved, false);
26 });
```

The second test reproduces a realistic concurrency concern: a vote is applied, a separate save succeeds, then the vote fails. Rolling back only vote fields must keep saved=true. This is more useful than asserting that a setter or reducer was merely called.

Validation boundary tests

The other two unit tests check malformed/trimmed email, required passwords, the seven/eight-character boundary, confirmation mismatch, deletion confirmation and the 30-character/200-character profile limits. They test the actual validator functions used by React forms.

Also inspect: tests/state.test.mjs, lines 28-38

Q. Unit test versus end-to-end test?
A. A unit test isolates small logic such as a reducer. An end-to-end test exercises a browser workflow with rendering, events and API behavior.

Q. Does passing a reducer test prove the button works?
A. No. Browser tests must also prove the click dispatches the right update, sends the request and displays success/failure correctly.

Browser and visual verification

Test group	What passed
12 existing interaction tests	Feed controls/history; votes/save/hide; post/comment editing and deletion; replies; draft/image posting; profiles/preferences; communities/custom feeds; chats/notifications; careers/press; login and signup.
4 new behavior tests	Controlled account validation/error/retry/reset; settings history/reload/fallback/live colors; failed preference rollback; failed vote rollback and saved state across layouts.
3 responsive tests	Each visits all 11 page entries at one width: 390, 768 or 1440px. Document and feed must fit horizontally.
1 sidebar-state test	A persisted collapsed-sidebar preference must not squeeze phone/tablet content.
4 unit tests	Immutable vote transitions, rollback isolation, account validation and profile limits.
52 screenshot comparisons	All recorded states have zero changed pixels at Pixelmatch threshold 0.1. Includes desktop pages plus selected guest/member, mobile-home and dark-mode states.

How visual comparison works

The runner freezes time, replays recorded GET responses and compares PNG pixels. It permits a changed-pixel ratio up to 0.001; the observed ratio was zero for all 52 cases. Separate screenshots were inspected for the newly fixed community/editor mobile and press tablet layouts.

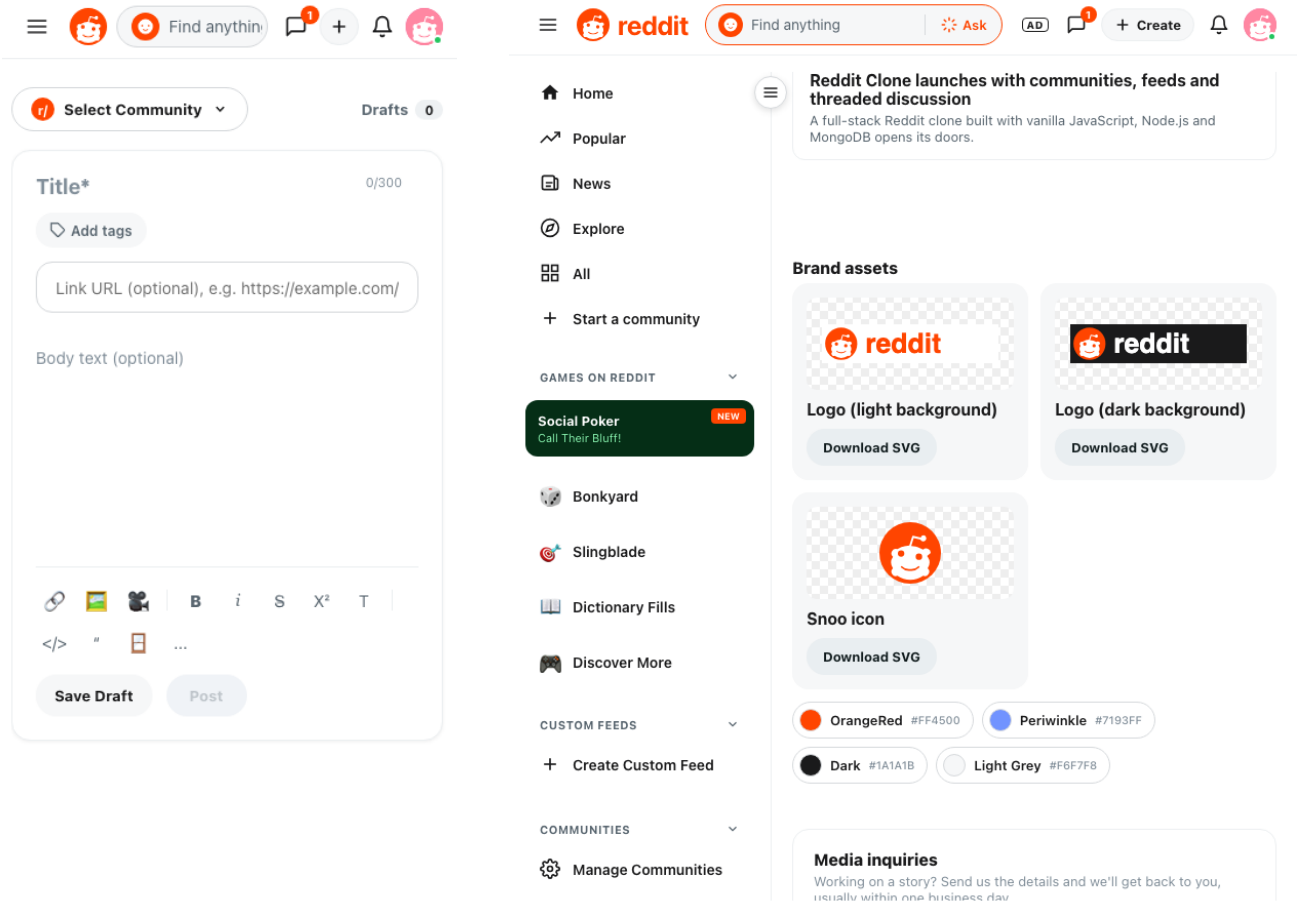
Source: tests/visual-parity.mjs, lines 73-93

State the limits honestly

Tests sample behavior; they do not prove every function in every browser. Full password-reset/code-login, video uploads, actual resume submission, account deletion, all permissions and cross-browser behavior remain incompletely tested. The account-validation browser test mocks email success/error responses, so it checks the UI contract without changing the demo account's email.

Q. Why use both visual and interaction tests?
A. Correct pixels do not prove an API action works, and a working click does not prove layout preservation. The checks cover different risks.

Visual evidence for the responsive fixes



Left: editor at 390px. Right: press assets at 768px.

The editor keeps every toolbar action accessible by wrapping onto another row. The press page uses two asset columns within the tablet content area. Neither solution simply hides overflowing content. The project's existing colors, typography, controls and assets are retained.

The mobile community/profile grid was also checked: the content uses the available single column instead of retaining a desktop sidebar track. The browser suite verifies document/feed widths across all page entries at the three representative sizes.

Screenshots captured from the local running application after the fixes. These images illustrate layout evidence; they do not replace the behavior tests.

Your eight-minute evaluation demonstration

Time	Demonstration	Explain while showing it
0:00-0:45	Introduce the app and architecture.	React/Vite, Express/API, MongoDB; what Context/Router do; honest hybrid scope.
0:45-2:00	Open a feed, switch card/compact and vote/save.	PostCard state; reusable VoteControls; props; immutable reducer; parent cache and keys.
2:00-3:00	Open Profile settings and change colors.	Controlled fields; FormSection/FormField; local preview state; dynamic inline styling; save callback.
3:00-4:00	Switch settings tabs, press Back/Forward and reload.	URL-derived tab state; useSearchParams; Context-preserving portal; route matching versus server routes.
4:00-5:15	Submit invalid email or mismatched passwords.	Validation before API calls; first-invalid-field focus; pending guard; server-error preservation and retry.
5:15-6:15	Comment, reply/edit and cancel.	Reusable controlled form, callback props, keyboard submit and conditional Cancel button.
6:15-7:00	Resize community/editor/press screens.	Responsive grid specificity, toolbar wrapping and tablet asset columns.
7:00-8:00	Open tests and explain one failure case.	Immutable vote rollback or preference rollback; what tests prove; actual personal contribution.

Prepare the environment

Start MongoDB and Express, sign into the local demo account, and keep the relevant source files open. The demo account is demo_user with password123. Use the test database for destructive experiments; the evaluation demo does not need to delete an account or reset the database.

Q. What if the evaluator interrupts with a question?
A. Pause the demonstration and trace the exact value/handler involved. Explain what happens before the request, during pending state and after success or failure.

Q. How do I prove understanding beyond memorization?
A. Predict a state transition, make one small safe change, run a focused test, and explain why it passed or failed. Use the exercises on the next page.

Debugging exercises you should solve yourself

Exercise 1 - The vote changes by two

Question: A post moves from your upvote to your downvote. Why does its score fall by two?
Check: Read postReducer. Start from score 10 and userVote=1. Predict the result before running the unit test.
Expected reasoning: remove the previous +1 contribution and apply -1; the new score is 8. The original object must remain unchanged.

Exercise 2 - A failed request must not lose another update

Question: A vote fails after Save succeeds. Which fields may be rolled back?
Check: Read the rollback unit test and the catch block in PostCard.
Expected reasoning: restore score/userVote only. Replacing the whole post with an old snapshot could erase saved=true.

Exercise 3 - The URL changes but the tab does not

Question: Why is a local tab variable disconnected from browser Back/Forward?
Check: Read Settings' useSearchParams call and derive tab from the current query.
Expected reasoning: browser navigation updates Router location; deriving the active tab from that location keeps rendering synchronized.

Exercise 4 - Mobile content still has desktop columns

Question: Why did a mobile .main-layout rule fail on full-width pages?
Check: Compare .main-layout with .main-layout.no-right-col in browser computed styles.
Expected reasoning: the modifier selector was more specific. Fix the applicable grid rule, not just overflow visibility.

Exercise 5 - Validation versus server rejection

Question: A correctly formatted email is rejected because the password is wrong. Is the field validator broken?
Expected reasoning: no. Client validation checks format/required rules; server authentication can still fail. Preserve values, show the server message and allow retry.

Practice discipline

For each exercise: predict the outcome, locate the source, inspect state/URL/network behavior, make a change only on a practice branch, run a focused test, then restore the intended implementation. Record your own observations rather than repeating an answer without reproducing it.

Your contribution and explanation worksheet

The rubric assesses your individual understanding and contribution, which generated code or a PDF cannot establish. This revision was prepared with assistant support. Explain which parts you personally designed, implemented, reviewed, tested or debugged; do not claim unreviewed generated changes as independent authorship.

Fill in your own evidence before the viva

Prompt	Your notes to prepare
Feature I can explain without notes	Name the page/component, its state, props, events and API endpoint.
Change I personally made or reviewed	Identify the exact file/function and why the change was needed.
Bug I reproduced	Describe the original behavior, input/viewport and observed failure.
How I diagnosed it	Name the evidence: state values, network request, computed style, console or failing test.
Fix and validation	Describe why the fix works and which test/demo verifies it.
Assistance I used	State what help you received and how you checked/understood the result.
Remaining limitation	Identify a real boundary such as retained template controllers or an untested workflow.

Explain a component in six sentences

1. Its responsibility is ...
2. It receives these props ...
3. It owns this state because ...
4. This event calls this handler ...
5. Success and failure produce these UI changes ...
6. This test checks the behavior, with these limits ...

A safe example to rehearse

Practice saying: VoteControls receives post and compact props and renders the same voting UI in two layouts. PostCard owns the interaction and post data. A click dispatches an immutable vote action and sends the API request. If the request fails, only vote fields are rolled back. The parent feed cache receives changes so layout switches keep the saved/vote data. Unit and browser tests cover the transition and failure behavior.

Use “my project” to identify the application you are presenting. Attribute your own authorship and contribution accurately according to your course rules.

Viva questions: components and styling

Q. What is component reuse in your code?
A. VoteControls and PostMenu are used in full and compact post layouts. FormSection/FormField are reused in account/profile forms; CommentComposer handles comments, replies and edits.

Q. What is JSX?
A. Syntax transformed into JavaScript element creation. Expressions go inside braces; DOM class is represented with className.

Q. Why are props read-only?
A. They are inputs from the parent. Mutating them breaks predictable ownership. The reducer creates new post objects and the parent updates its list immutably.

Q. How does a child communicate upward?
A. The parent passes a callback prop. PostCard calls onChange with changed fields; FormSection calls its supplied onSubmit with validated values.

Q. What is a render callback?
A. A function prop returning UI. FormSection calls children(form) so its caller can render fields using the shared form state and handlers.

Q. Does a rerender replace the whole DOM?
A. No. React compares new element descriptions with the previous tree and commits needed changes. Keys help preserve sibling identity.

Q. What happens when the view is part of the key?
A. Switching card/compact creates a new card instance. The parent cache preserves changed post data while local menu/media state resets.

Q. How is conditional rendering used?
A. Joined communities omit JoinButton; menus depend on login/permissions; cards choose full/compact; settings chooses a tab or login prompt; feed status chooses loading/error/empty/end content.

Q. CSS hiding versus conditional removal?
A. CSS hiding keeps content mounted. Conditional rendering changes rendered content. Component Effects persist until that component actually unmounts.

Q. What are dynamic styles here?
A. Profile banner/avatar colors depend on controlled state. Selected swatch/tab classes and vote classes depend on current values. Toast opacity/transform depend on visibility state.

Q. How did you preserve the look?
A. Retained the existing CSS classes, structure, icons and assets while converting behavior/markup to React. Baseline comparisons verify the recorded states; responsive fixes address extra narrow layouts.

Q. Why not hide overflow to pass the test?
A. Hidden overflow can make controls inaccessible. The fixes let grids and toolbar content actually fit/wrap.

Viva questions: Hooks, routing and forms

Q. useState versus useRef?
A. State updates schedule rendering. Ref assignments do not. Use state for text/pending UI and refs for DOM nodes or synchronous duplicate guards.

Q. useState versus useReducer?
A. useState is convenient for simple independent values. A reducer centralizes related transitions such as vote and patch updates, and can be tested as a pure function.

Q. useMemo versus useCallback?
A. useMemo caches a calculated result, such as parsed markup. useCallback caches a function reference, such as loadMore or the settings initializer.

Q. Why do Effects need cleanup?
A. Listeners, observers, subscriptions and timers can outlive one render. Cleanup releases them before replacement or unmount.

Q. What is a stale closure?
A. A callback can capture values from an older render. Correct dependencies, functional setters and refs help avoid using outdated data.

Q. What is Context used for?
A. Shared session data. It does not replace the server cookie/database and does not need to own all local UI state.

Q. What is a controlled checkbox?
A. Its checked prop comes from state and onChange writes event.target.checked back to state. Preferences uses this pattern through PreferenceToggle.

Q. Why validate on the server as well?
A. Requests can bypass the frontend. Server validation and authorization are necessary even when the UI blocks invalid input.

Q. What is push versus replace navigation?
A. Push adds a history entry; replace edits the current entry. Settings tabs push so Back/Forward revisit tabs; feed sort updates replace the current query entry.

Q. Why use a portal for settings?
A. It renders into the existing feed slot while keeping the root Router/Auth Context ancestry. An independent createRoot would start a separate Context tree.

Q. Can we promise every workflow is bug-free?
A. No. Build, unit, browser and screenshot checks provide evidence for covered cases. Unusual inputs, other browsers and untested integrations still require validation.

Q. What must I demonstrate personally?
A. Trace a feature, predict a state transition, diagnose a failure, explain tests and state your actual contribution. The guide supports practice; it cannot perform the viva for you.

Function types used in the project

React applications use ordinary JavaScript functions in several roles. Function component describes a rendering responsibility; async describes Promise behavior; arrow function describes syntax. One function can belong to more than one category.

Kind	Actual example and meaning
Function component	VoteControls, PostMenu, FormSection and SettingsPage return renderable React content.
Custom Hook	useForm combines useState/useRef into reusable form behavior; useAuth reads Context; usePage initializes pages.
Event handler	PostCard.action receives a click event; useForm.submit receives the form submission event.
Arrow callback	FormField onChange passes event.target.value to form.change. Settings buttons call setParams.
Async function	PostCard.vote and useForm.submit await API/callback work and handle errors.
Functional state updater	Feed uses a current => current.map(...) callback to replace one cached post immutably.
Reducer	postReducer computes next post data from previous data and a vote/patch action.
Effect setup / cleanup	Feed sets up an observer and returns a disconnect function; the click listener returns its removal cleanup.
Callback prop / render callback	onSubmit passes behavior to a form; children(form) lets its caller render fields using shared state.
Validator / utility	validateAccount returns field errors without side effects; formatCount and postUrl format values/URLs.
Store method	authState.setUser replaces the session snapshot and notifies subscribers. It is not a React Hook.

Q. onClick={save} versus onClick={save()}?
A. The first passes a function to run on the click. The second calls save during rendering and passes its return value. Use an arrow callback when arguments are needed.

Q. preventDefault versus stopPropagation?
A. The first prevents browser default behavior, such as form navigation. The second stops propagation to other handlers. Feed dropdown clicks stop propagation so the document menu-closing listener does not immediately close them.

React click-handler example: frontend/react/Feed.jsx, lines 20-25

Q. Is every callback a Hook?
A. No. A Hook is a React-aware API following Hook calling rules. Ordinary event or array callbacks must not call state/effect/context Hooks inside themselves.

Source index: open these files first

Root / providers: frontend/react/main.jsx, lines 7-7

Page routes and navigation bridge: frontend/react/App.jsx, lines 21-29

Reusable voting component: frontend/react/PostContent.jsx, lines 12-19

Conditional menu component: frontend/react/PostContent.jsx, lines 28-44

Optimistic vote and rollback: frontend/react/PostCard.jsx, lines 34-45

Pure reducer: frontend/react/postState.mjs, lines 2-8

Feed state and refs: frontend/react/Feed.jsx, lines 31-43

Fetching Effect: frontend/react/Feed.jsx, lines 64-68

Context provider and consumer: frontend/react/AuthContext.jsx, lines 10-19

URL-driven tab selection: frontend/react/SettingsPage.jsx, lines 127-139

Portal and page startup: frontend/react/SettingsPage.jsx, lines 141-147

Controlled field: frontend/react/forms/FormSection.jsx, lines 4-12

Validation and submission: frontend/react/forms/useForm.js, lines 13-32

Validation rules: frontend/react/forms/validation.mjs, lines 1-13

Comment submission: frontend/react/CommentComposer.jsx, lines 17-32

Mobile grid: frontend/css/responsive.css, lines 28-38

New browser checks: tests/rubric.spec.mjs, lines 24-26

Immutable-state tests: tests/state.test.mjs, lines 6-18

How to read the source excerpts

The code gutters show actual source line numbers. A visually wrapped continuation has an empty gutter. The links target the exact Git commit, so later changes to main will not silently change the code you are studying.

The guide quotes selected relevant blocks. Open the linked file for surrounding logic, imports, callbacks and cleanup rather than treating an excerpt as a standalone program.

References, verification and scope

Theory was checked against primary documentation. Project implementation statements come from the pinned source. The supplied rubric defines the evaluation targets; tests provide evidence for covered behavior, not a self-awarded mark.

React overview, components, props and events

State and functional updaters

Updating objects without mutation

Conditional rendering

Effects and cleanup

Context and providers

External store subscriptions

Refs

Memoized calculations

Memoized callbacks

Reducers

Controlled input behavior

Portals preserve React ancestry

React Router declarative routing

React Router query parameters

Vite deployment

Playwright browser testing

Snapshot and recorded validation

Commit: 60e7ea999980c6400886034209ea8e3c4e38c2b0. GitHub main was verified against this commit after pushing. Production build and whitespace checks passed. The current validation recorded 20 passing browser tests, four passing unit tests, and 52 screenshot comparisons with zero changed pixels and no recorded page errors.

Limits of the implementation claim

The repository now has direct evidence for the technical rubric topics. It remains a hybrid migration, with some native/template controllers and document navigation outside the React settings workflow. Public deployment, every edge case and every browser have not been established. Personal understanding and contribution must be demonstrated by you.

This edition supersedes the earlier guide: PostCard no longer uses a forced-refresh counter or mutable props, and settings is no longer described as native uncontrolled forms.